



Projet Calcul Scientifique

Mathis Magron
Lilian Desesquelles

Année universitaire 2024/2025

Table des matières

1	Introduction	2
2	Limitations of the power method	2
2.1	Question 1 :	3
2.2	Question 2 :	3
2.3	Question 3 :	4
3	Extending the power method to compute dominant eigenspace vectors	4
3.1	subspace_iter_v0 : a basic method to compute a dominant eigenspace	4
3.1.1	Question 4 :	4
3.1.2	Question 5 :	4
3.1.3	Question 6 :	5
3.2	subspace_iter_v1 : improved version making use of Raleigh-Ritz projection	6
3.2.1	Question 7 :	6
4	subspace_iter_v2 and subspace_iter_v3 : toward an efficient solver	7
4.1	Block approach (subspace_iter_v2)	7
4.1.1	Question 8 :	7
4.1.2	Question 9 :	7
4.1.3	Question 10 :	7
4.2	Deflation method (subspace_iter_v3)	8
4.2.1	Question 11 :	8
4.2.2	Question 12 :	8
4.2.3	Question 13 :	8
5	Numerical experiments	9
5.1	Question 14 :	9
5.2	Question 15 :	10
6	Part 2 : Application to Image Compression	10
6.1	Question 1 :	10
6.2	Question 2 :	11

1 Introduction

This project, carried out as part of the Calcul Scientifique course, focuses on the study and implementation of the Subspace Iterative Method. The goal is to understand the limitations of the classical power method and to explore more efficient alternatives, particularly subspace iteration methods.

In the first part of the project, we began by evaluating the performance and limitations of the basic power method and its deflation-based extension. We then progressively implemented and analyzed several variants of the subspace iteration method, each introducing improvements in terms of convergence speed,

The algorithms developed in this phase will be applied in the second session to compress images, thus demonstrating their practical utility in real-world applications.

PS : We will only be answering the questions of the initial subject without reminding them.

2 Limitations of the power method

Here is a quick reminder of the power method algorithm :

Algorithm 1 Vector power method

Input : Matrix $A \in \mathbb{R}^{n \times n}$

Output : (λ_1, v_1) eigenpair associated to the largest (in module) eigenvalue.

$v \in \mathbb{R}^n$ given

$\beta = v^T \cdot A \cdot v$

repeat

$y = A \cdot v$

$v = y / \|y\|$

$\beta_{old} = \beta$

$\beta = v^T \cdot A \cdot v$

until $|\beta - \beta_{old}| / |\beta_{old}| < \varepsilon$

$\lambda_1 = \beta$ and $v_1 = v$

2.1 Question 1 :

TABLE 1 – Comparaison des temps d'exécution entre `power_v11` et `eig`.

Taille de la Matrice	Type de Matrice	Temps itéré (s)	Temps eig (s)
100x100	1	0.000	3.00×10^{-2}
10x10	1	0.000	1.00×10^{-2}
500x500	1	7.00×10^{-2}	1.70×10^1
100x100	2	0.000	1.00×10^{-2}
10x10	2	0.000	0.000
500x500	2	3.00×10^{-2}	7.00×10^{-2}
100x100	3	0.000	1.00×10^{-2}
10x10	3	0.000	0.000
500x500	3	7.00×10^{-2}	2.70×10^{-1}
100x100	4	0.000	3.00×10^{-2}
10x10	4	0.000	1.00×10^{-2}
500x500	4	7.00×10^{-2}	1.76×10^1

Commentaire : La méthode `power_v11` est généralement plus rapide que la méthode `eig`.

2.2 Question 2 :

Réponse : Pour réduire le nombre de produits matrice $\times$ vecteur dans la boucle, nous pouvons réorganiser les opérations de manière à calculer $A \cdot v$ une seule fois par itération.

Algorithm 2 Vector power method

Input : Matrix $A \in \mathbb{R}^{n \times n}$

Output : (λ_1, v_1) eigenpair associated to the largest (in module) eigenvalue.

$v \in \mathbb{R}^n$ given

$\beta = v^T \cdot A \cdot v$

repeat

$v = z / \|z\|$

$z = A \cdot v$

$\beta_{old} = \beta$

$\beta = v^T \cdot A \cdot v$

until $|\beta - \beta_{old}| / |\beta_{old}| < \varepsilon$

$\lambda_1 = \beta$ and $v_1 = v$

TABLE 2 – Comparaison des temps d'exécution entre `power_v12` et `eig`.

Taille de la Matrice	Type de Matrice	Temps itéré (s)	Temps eig (s)
100x100	1	1.00×10^{-2}	2.00×10^{-2}
10x10	1	0.000	1.00×10^{-2}
500x500	1	4.00×10^{-2}	9.05
100x100	2	1.00×10^{-2}	0.000
10x10	2	0.000	1.00×10^{-2}
500x500	2	5.00×10^{-2}	6.00×10^{-2}
100x100	3	0.000	0.000
10x10	3	0.000	1.00×10^{-2}
500x500	3	6.00×10^{-2}	2.60×10^{-1}
100x100	4	1.00×10^{-2}	2.00×10^{-2}
10x10	4	0.000	0.000
500x500	4	8.00×10^{-2}	9.35

On peut donc voir que l'algorithme `v12` est beaucoup plus rapide que celui de `v11`.

2.3 Question 3 :

Réponse : L'inconvénient de la méthode de déflation est que l'on recalcule les vecteurs propres restants après chaque déflation. Cela prend donc plus de temps de calcul, surtout pour les grandes matrices qui vont demander beaucoup de temps de calcul.

3 Extending the power method to compute dominant eigenspace vectors

3.1 `subspace_iter_v0` : a basic method to compute a dominant eigenspace

3.1.1 Question 4 :

Si nous appliquons la méthode de puissance à un ensemble de m vecteurs sans orthonormaliser, la matrice V ne convergera pas vers une matrice des m vecteurs propres distincts de A . Au lieu de cela, les vecteurs vont converger vers le vecteur propre dominant (associé à la plus grande valeur propre en module). C'est pourquoi une orthonormalisation est nécessaire pour éviter cette convergence vers un seul vecteur propre.

3.1.2 Question 5 :

Réponse : La matrice H est de taille $m \times m$, alors que A est de taille $n \times n$ où m est très souvent beaucoup plus petit que n , donc le calcul de

la décomposition spectrale complète de H prend moins de temps que la décomposition spectrale complète de A .

3.1.3 Question 6 :

Algorithm 3 Subspace Iteration Algorithm

```

Compute the norm of  $A$  :  $\text{normA} \leftarrow \|A\|$ 
 $n \leftarrow \text{size}(A, 1)$ 
Initialize convergence indicator :  $\text{conv} \leftarrow 0$ 
Initialize current iteration number :  $k \leftarrow 0$ 
Generate an initial set of  $m$  orthogonal vectors :  $V \leftarrow \text{mgs}(\text{randn}(n, m))$ 
while  $\neg \text{conv} \wedge k < \text{maxit}$  do
     $k \leftarrow k + 1$ 
    Compute  $Y = A \cdot V$ 
    Compute the Rayleigh quotient  $H = V^T \cdot Y$ 
    Check convergence :  $\text{conv} \leftarrow (\|Y - V \cdot H\|_F \leq \epsilon \cdot \text{normA})$ 
    Orthogonalize :  $V \leftarrow \text{mgs}(Y)$ 
end while
Perform spectral decomposition of  $H$  :  $[V_H, D_H] \leftarrow \text{eig}(H)$ 
Sort eigenvalues in descending order :  $[W, I] \leftarrow \text{sort}(\text{diag}(D_H), \text{'descend'})$ 
Permute eigenvectors accordingly :  $X \leftarrow V_H(:, I)$ 
Compute the  $m$  dominant eigenvectors of  $A$  :  $V \leftarrow V \cdot X$ 
 $D \leftarrow \text{diag}(W)$ 
 $\text{it} \leftarrow k$ 
if  $\text{conv}$  then
     $\text{flag} \leftarrow 0$ 
else
     $\text{flag} \leftarrow -3$ 
end if

```

3.2 subspace_iter_v1 : improved version making use of Raleigh-Ritz projection

3.2.1 Question 7 :

Algorithm 4 Raleigh-Ritz projection

Input : Matrix $A \in \mathbb{R}^{n \times n}$ and an orthonormal set of vectors V .

Output : The approximate eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .

Compute the Rayleigh quotient $H = V^T \cdot A \cdot V$.

Compute the spectral decomposition $X \cdot \Lambda_{out} \cdot X^T = H$, where the eigenvalues of H ($diag(\Lambda_{out})$) are arranged in descending order of magnitude.

Compute $V_{out} = V \cdot X$.

The algorithm of the subspace_iter_v1 is then :

Algorithm 5 Subspace iteration method v1 with Raleigh-Ritz projection

Input : Symmetric matrix $A \in \mathbb{R}^{n \times n}$, tolerance ε , *MaxIter* (max nb of iterations) and *PercentTrace* the target percentage of the trace of A

Output : n_{ev} dominant eigenvectors V_{out} and the corresponding eigenvalues Λ_{out} .

Generate an initial set of m orthonormal vectors $V \in \mathbb{R}^{n \times m}$; $k = 0$;
PercentReached = 0

repeat

$k = k + 1$

 Compute Y such that $Y = A \cdot V$ (Compute the Rayleigh quotient)

$V \leftarrow$ orthonormalisation of the columns of Y (**Compute the Rayleigh quotient**)

Rayleigh-Ritz projection applied on matrix A and orthonormal vectors V (**Compute the spectral decomposition**)

Convergence analysis step : save eigenpairs that have converged and update *PercentReached* (**Compute $V_{out} = V \cdot X$**)

until (*PercentReached* > *PercentTrace* or $n_{ev} = m$ or $k > \text{MaxIter}$)

4 subspace_iter_v2 and subspace_iter_v3 : toward an efficient solver

4.1 Block approach (subspace_iter_v2)

4.1.1 Question 8 :

Réponse : L'algorithme de multiplication matricielle a une complexité de $O(n^3)$, donc pour calculer A^p on a une complexité de $O(pn^3)$ donc un total de pn^3 flops alors que le calcul de $A^p \cdot V$ est de $O(n^2m)$, où m est le nombre de colonnes de V .

```
Y = V;  
for i = 1:p  
    Y = A * Y;  
end
```

Cet algorithme permet de réduire le nombre de flops à $O(n^2mp)$ ce qui est plus efficace.

4.1.2 Question 9 :

Réponse : Voici une implémentation de l'accélération dans `subspace_iter_v2.m` :

```
k = k + 1;  
Y = Vr;  
for i = 1 :p do  
    Y = A * Y;  
end for  
Vr = mgs(Y);  
  
[Wr, Vr] = rayleigh_ritz_projection(A, Vr);  
  
analyse_cvg_finie = 0;  
  
nbc_k = 0;  
  
i = nb_c + 1;
```

4.1.3 Question 10 :

Réponse : Lorsque p augmente, on converge plus rapidement car à chaque itération on effectue plus de multiplications par A , on accélère donc la convergence vers les vecteurs propres élevés. Il ne faut néanmoins pas prendre une valeur de p trop élevée car ça peut entraîner une perte de précision et une augmentation du temps de calcul.

4.2 Deflation method (subspace_iter_v3)

4.2.1 Question 11 :

Réponse : Avec la méthode `subspace_iter_v1`, Les vecteurs propres associés aux plus grandes valeurs propres convergent plus rapidement que celle associées aux plus petites valeurs propres. On aura donc une précision qui décroît avec le module des valeurs propres, les valeurs les plus grandes seront celles les plus précises.

4.2.2 Question 12 :

Réponse : Avec la méthode `subspace_iter_v3`, la précision des couples propres sera plus grande globalement car on va figés les vecteurs propres convergents, on aura donc moins d'erreurs.

4.2.3 Question 13 :

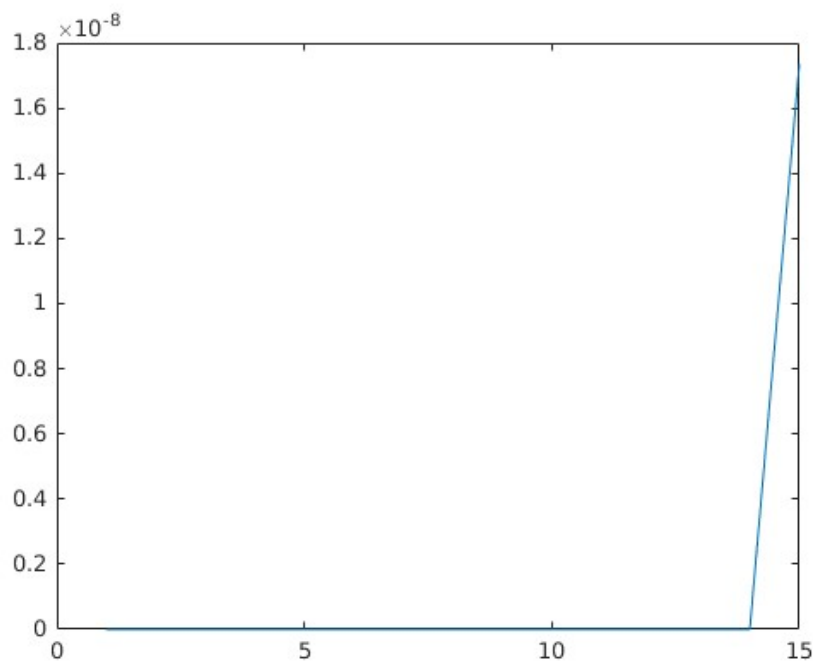
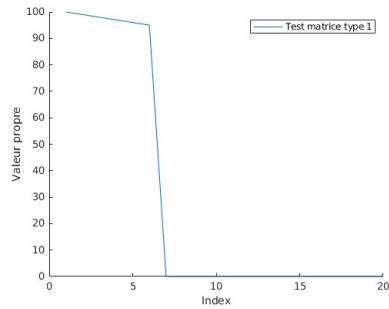


FIGURE 1 – Temps d'exécution minimum du programme en fonction de p

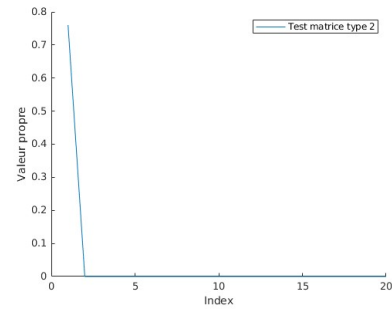
Réponse : Nous pouvons voir que le temps augmente lorsque p est trop grand.

5 Numerical experiments

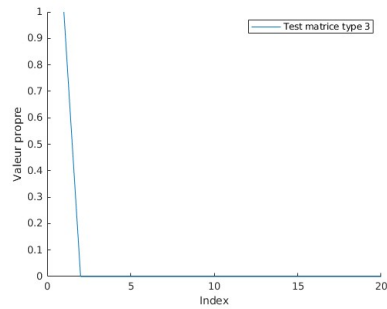
5.1 Question 14 :



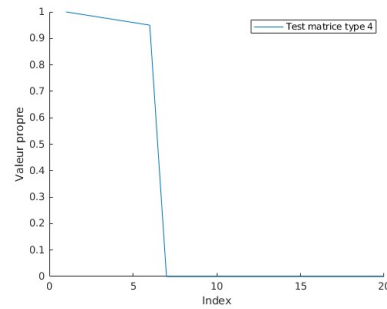
(a) Test matrice type 1



(b) Test matrice type 2



(c) Test matrice type 3



(d) Test matrice type 4

FIGURE 2 – Figures montrant la distribution des valeurs propres des différents types de matrices

Veuillez vous référer au code Matlab pour l'implémentation des figures.

5.2 Question 15 :

TABLE 3 – Comparaison des temps d'exécution entre les différentes méthodes.

Taille de la Matrice	Type de Matrice	Temps v0	Temps v1
100x100	1	0.61	0.34
300x300	1	5.10	2.03
500x500	1	31.00	NaN
100x100	2	0.04	0.02
300x300	2	1.50	0.08
500x500	2	0.58	0.10
100x100	3	0.16	0.02
300x300	3	0.61	0.09
500x500	3	1.95	0.24
100x100	4	0.68	0.26
300x300	4	5.60	1.98
500x500	4	27.00	NaN

Temps v2	Temps v3	Temps v11	Temps v12
0.17	0.03	0.03	0.03
1.93	0.17	NaN	13.3
NaN	NaN	NaN	153
0.03	0.03	NaN	0,0
0.07	0.02	NaN	0.12
0.13	0.11	NaN	0.23
0.03	0.01	NaN	0.0
0.11	0.06	NaN	0.34
0.26	0.15	NaN	1.42
0.28	0.05	NaN	0.02
2.05	0.18	NaN	13.1
NaN	NaN	NaN	128.7

6 Part 2 : Application to Image Compression

6.1 Question 1 :

Réponse :

1. $\Sigma_k \in \mathcal{M}_k(\mathbb{R})$
2. $U_k \in \mathcal{M}_{q,k}(\mathbb{R})$
3. $V_k \in \mathcal{M}_{p,k}(\mathbb{R})$

Et si $q < p$,

1. $\Sigma_k \in \mathcal{M}_k(\mathbb{R})$
2. $U_k \in \mathcal{M}_{p,k}(\mathbb{R})$
3. $V_k \in \mathcal{M}_{q,k}(\mathbb{R})$

6.2 Question 2 :

Réponse :

Voici tout d'abord les différents temps d'exécution obtenu pour chaque algorithmes :

1. v0 : Le temps écoulé est de 32.756845 seconds.
2. v1 : Le temps écoulé est de 3.209248 seconds.
3. v2 : Le temps écoulé est de 3.356092 seconds.
4. v3 : Le temps écoulé est de 1.844697 seconds.
5. eig : Le temps écoulé est de 0.089363 seconds.

Nous pouvons noter que tout les algorithmes ne se valent pas en terme de temps d'exécutions : il est important de remarquer que l'utilisation d'eig est à prioriser tandis que celle de subspace_iter_0 est à éviter.

La compression d'images marche aussi bien pour des images en format portraits que paysages.

La RMSE reste la même quelque soit l'algorithme utilisé :

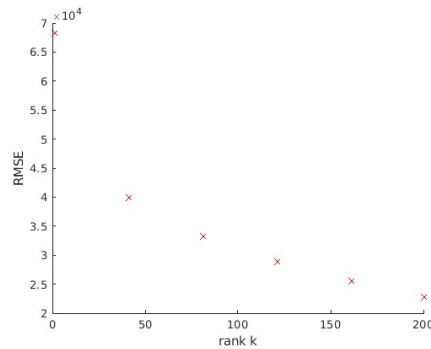
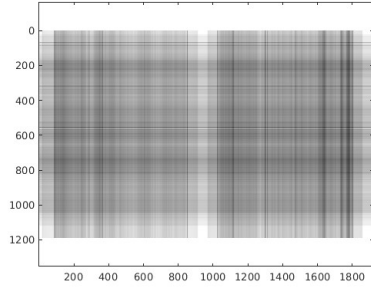
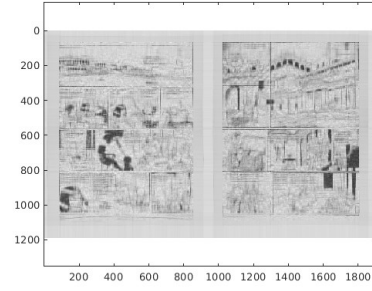


FIGURE 3 – RMSE en fonction du rang k pour une image en paysage

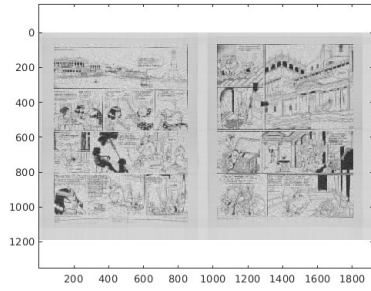
Pour aller plus loin nous avons essayé d'implémenter la reconstruction d'images en couleur mais nous avons eu des soucis dans l'implémentation de 3 SVD, une par canaux de couleur R G B (code disponible dans le fichier ReconstructionImage_couleur.m), vous trouverez donc en dessous un exemple de ce que produit l'algorithme pour une image en niveau de gris.



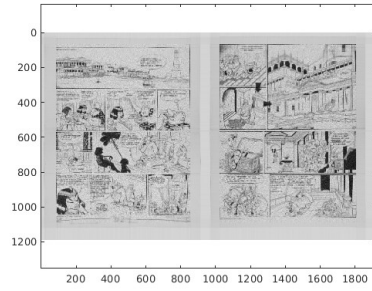
(a) Itération 1



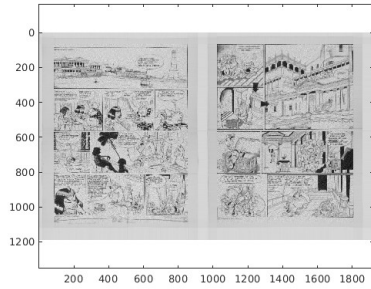
(b) Itération 2



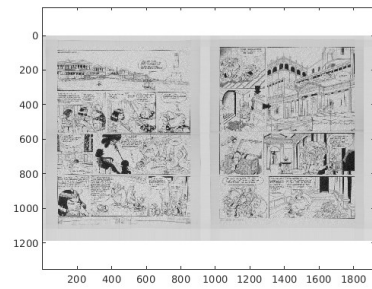
(c) Itération 3



(d) Itération 4



(e) Itération 5



(f) Itération 6

FIGURE 4 – Affichage des différentes images reconstruites au cours de l'algorithme des meilleures approximations de rang faible